

ELEC3609 - Individual Task 7

Understand and Evaluate Django Security | 6 October 2026

Task 1 - Django Security

1. When and why is a CSRF token needed?

A valid CSRF token is needed for requests that change data, such as POST, PUT and DELETE, especially when the browser automatically sends session cookies. It helps Django distinguish a request made through the application from one forged by another website. For example, an email-change POST form needs Django's CSRF token so a malicious page cannot change a logged-in user's address.

2. What is the difference between HttpOnly and Secure cookies?

Without HttpOnly, injected JavaScript from an XSS attack can read a session cookie and potentially steal the user's session. Without Secure, the browser may send the cookie over unencrypted HTTP, allowing interception on the network. HttpOnly restricts script access, while Secure restricts transmission to HTTPS; both should protect session cookies, but they do not replace XSS or CSRF protection.

3. What happens if CSRF middleware is disabled?

If CSRF middleware is disabled without equivalent protection, manually validating form fields does not stop forged requests. Hidden cross-site forms can change account details when the browser sends the victim's session cookies, or force the browser to log into an attacker-controlled account (login CSRF). I would keep CsrfViewMiddleware enabled because it checks the token and the request's Origin or, for HTTPS, Referer.

4. How should passwords be protected in Django?

Django stores a salted, slow password hash rather than plaintext, using PBKDF2 with SHA-256 by default, which makes offline guessing harder. I would use `set_password()` and Django's authentication helpers, and apply password validators in every password creation or change flow. I would also encourage long, unique passwords, use HTTPS and login rate limiting, and never store or log plaintext passwords.

5. Why is hiding an admin link insufficient?

Hiding a link does not enforce access control because a user can request the URL or its API directly. The backend must authenticate the user and check the required admin permission on every request, including supporting API endpoints. I would use `permission_required(..., raise_exception=True)` or `PermissionRequiredMixin` to deny unauthorised access; `login_required` alone only checks whether a user is logged in.

Task 2 - Project Progress Check-in

1. Current role

I work on both the frontend and backend of StudyCrew, particularly the connection between the profile interface and team data.

2. Features I am responsible for

My two main areas are the Profile page and consistent avatar display in project Team member lists.

3. Progress update

I have improved the Profile page and connected uploaded avatars to the Team member list. I also added image fallbacks, refreshed related data after profile updates, and added regression tests for avatar display and synchronisation.

4. Estimated completion

The project has already reached 90% completion. I plan to finish all code optimisation and testing by mid-October 2026.

5. Team feedback or concerns

My main concern is allowing enough time for shared integration testing and deployment checks. Clear ownership of the remaining tasks and keeping test results in one place would help avoid duplicated work and missed handover details.